

پروژه سوم درس نرم افزار ریاضی

Numerical Methods

علی تقی زاده

شماره دانشجویی:

۴۰۳۱۳۰۰۸

لینک مخزن گیت هاب:

github.com/alitkbb1/Numerical-Methods

۲۹ خرداد ۱۴۰۵

سوال ۱: درونیابی لاگرانژ و اسپلاین

در این بخش، هدف این است که برای مجموعه نقاط زیر، تابعی پیدا کنیم که از همه آن‌ها عبور کند:

$$(1, 1), (2, 3), (3, 5), (4, 8), (5, 5), (6, 2)$$

برای این کار از دو روش درونیابی لاگرانژ و اسپلاین مکعبی (Cubic Spline) استفاده شده است. خروجی برنامه نشان می‌دهد که هر دو روش درست پیاده‌سازی شده‌اند، چون مقدار خطا در گره‌های داده‌شده برابر صفر است و هر دو تابع دقیقاً از نقاط اصلی عبور می‌کنند.

نحوه پیاده‌سازی

برای درونیابی این شش نقطه، دو روش اصلی بررسی شد:

۱. **چندجمله‌ای لاگرانژ:** در این روش از تابع `scipy.interpolate.lagrange` استفاده شد. این تابع یک چندجمله‌ای یکتا می‌سازد که از تمام نقاط داده‌شده عبور می‌کند. چون در روش لاگرانژ فقط یک چندجمله‌ای برای کل بازه ساخته می‌شود، خروجی نهایی یک تابع سراسری و هموار است؛ البته همین ویژگی گاهی باعث ایجاد نوسان‌های ناخواسته، مخصوصاً در نزدیکی انتهای بازه، می‌شود.

۲. **اسپلاین مکعبی طبیعی:** برای این بخش از کلاس `CubicSpline` استفاده شد. در اسپلاین، به جای ساختن یک چندجمله‌ای واحد برای کل نقاط، بین هر دو نقطه متوالی یک چندجمله‌ای مکعبی جداگانه تعریف می‌شود. به همین دلیل، تابع نهایی به صورت تکه‌ای ساخته می‌شود و معمولاً رفتار طبیعی‌تر و کنترل‌شده‌تری بین نقاط دارد.

اعتبارسنجی نتایج: برای اطمینان از درست بودن پیاده‌سازی، مقدار هر دو تابع در نقاط اصلی داده بررسی شد. نتیجه نشان داد که هر دو روش دقیقاً از نقاط داده عبور می‌کنند. همچنین با کمک `sympy` فرم نمادین توابع نیز استخراج شد تا علاوه بر نمودار، نمایش تحلیلی آن‌ها هم در دسترس باشد.

نمایش و مقایسه: در پایان، منحنی هر دو روش همراه با نقاط اصلی رسم شد. این مقایسه کمک می‌کند تفاوت رفتار دو روش بهتر دیده شود: لاگرانژ یک چندجمله‌ای یکتا و سراسری می‌سازد، در حالی که اسپلاین با ساختار تکه‌ای خود معمولاً نوسان کمتری دارد و در بسیاری از داده‌های واقعی انتخاب مطمئن‌تری است.

۱. ضابطه توابع به دست آمده

الف) درونیابی لاگرانژ

از آنجا که ۶ نقطه داده شده است، چندجمله‌ای لاگرانژ از درجه ۵ خواهد بود. پس از محاسبه و ساده‌سازی، ضابطه چندجمله‌ای به شکل زیر به دست آمد:

$$P(x) = 0.175x^5 - 2.958x^4 + 18.375x^3 - 52.041x^2 + 68.45x - 31.0 \quad (1)$$

ب) درونیابی اسپلاین مکعبی

در روش اسپلاین مکعبی، تابع نهایی از چندجمله‌ای‌های درجه ۳ تشکیل می‌شود. این چندجمله‌ای‌ها در گره‌ها به هم متصل می‌شوند و علاوه بر مقدار تابع، مشتق اول و دوم آن‌ها نیز در نقاط اتصال پیوسته است. به عنوان نمونه، ضابطه تابع در بازه اول، یعنی $[1, 2]$ ، به صورت زیر محاسبه شد:

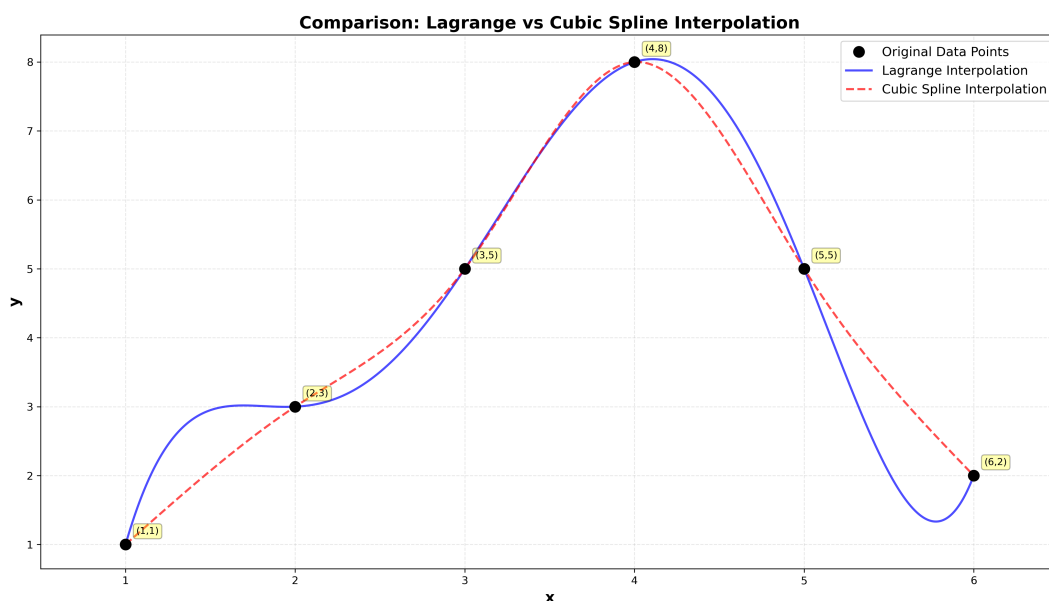
$$S_0(x) = -0.186x^3 + 0.559x^2 + 1.626x - 1.0 \quad (2)$$

ضابطه سایر بازه‌ها نیز به طور کامل در خروجی برنامه درج شده است.

۲. تحلیل نمودار و مقایسه دو روش

با توجه به نمودار شکل ۱، تفاوت رفتار دو روش کاملاً مشخص است:

- رفتار لاگرانژ: منحنی لاگرانژ، که با رنگ آبی نمایش داده شده، یک چندجمله‌ای درجه ۵ است. این منحنی از همه نقاط عبور می‌کند، اما به دلیل سراسری بودن چندجمله‌ای، در بعضی قسمت‌ها دچار نوسان بیشتری می‌شود. این مسئله به‌ویژه در بازه انتهایی بین $x = 5$ و $x = 6$ بیشتر دیده می‌شود. چنین رفتاری نمونه‌ای از پدیده رونگه (Runge's Phenomenon) در درونیابی چندجمله‌ای است.
- رفتار اسپلاین: در مقابل، اسپلاین مکعبی که با خط چین قرمز نشان داده شده، مسیر نرم‌تر و طبیعی‌تری بین نقاط طی می‌کند. چون این روش به صورت تکه‌ای عمل می‌کند و در هر بازه فقط از یک چندجمله‌ای درجه ۳ استفاده می‌شود، نوسان‌های کاذب کمتری ایجاد می‌کند. به همین دلیل، برای چنین داده‌هایی معمولاً اسپلاین انتخاب مناسب‌تری است.



شکل ۱: مقایسه درونیابی لاگرانژ و اسپلاین مکعبی روی نقاط داده‌شده

سوال ۲: مصورسازی و تحلیل زمان اجرای الگوریتم‌ها

در این بخش، زمان اجرای سه الگوریتم (Alg.1, Alg.2, Alg.3) برای مجموعه داده‌هایی با اندازه ۱۰۰ KB تا ۷۰۰ KB بررسی شده است. داده‌ها از فایل اکسل خوانده شده‌اند و ساختار کد به صورتی نوشته شده که با اضافه شدن داده جدید، مانند سطر مربوط به ۷۰۰ KB، بدون نیاز به تغییر جدی در برنامه، محاسبات و نمودارها به‌روزرسانی شوند.

نحوه پیاده‌سازی

در این سوال، داده‌های مربوط به زمان اجرای الگوریتم‌ها مستقیماً از فایل اکسل خوانده شدند. برای این کار از تابع `pandas.read_excel` استفاده شد تا اطلاعات به شکل منظم وارد برنامه شوند. خواندن سطرها و ستون‌ها نیز به صورت پویا انجام شد تا کد در برابر اضافه شدن داده‌های جدید انعطاف‌پذیر باشد.

مصورسازی داده‌ها: برای اینکه رفتار الگوریتم‌ها بهتر قابل مقایسه باشد، سه نوع نمودار تولید شد:

- **نمودار میله‌ای:** برای مقایسه مستقیم زمان اجرای الگوریتم‌ها در اندازه‌های مختلف داده
- **نمودار خطی:** برای مشاهده روند رشد زمان اجرا و بررسی رفتار کلی هر الگوریتم
- **نمودار جعبه‌ای:** برای بررسی پراکندگی، پایداری و تفاوت آماری زمان اجرای الگوریتم‌ها

محاسبات آماری: برای هر الگوریتم، شاخص‌های آماری اصلی شامل میانگین، میانه و انحراف معیار محاسبه شد. همچنین میانگین زمان اجرای الگوریتم ۲ در بازه ۱۰۰ KB تا ۶۰۰ KB به صورت جداگانه به دست آمد. این محاسبه با فیلتر کردن `DataFrame` بر اساس اندازه داده انجام شد.

در مجموع، کد به گونه‌ای طراحی شده که هم در بخش پردازش داده و هم در بخش رسم نمودارها، وابستگی کمی به تعداد رکوردها داشته باشد و بتواند با ورودی‌های جدید نیز به خوبی کار کند.

الف و ب) رسم نمودارها و تحلیل رفتار الگوریتم‌ها

با توجه به ۷ رکورد موجود در داده‌ها، رفتار سه الگوریتم به صورت زیر قابل تحلیل است:

- **الگوریتم Alg.1:** این الگوریتم بهترین عملکرد را در میان سه الگوریتم دارد. زمان اجرای آن برای داده ۱۰۰ KB برابر ۵۰ ms است و حتی برای بزرگ‌ترین ورودی، یعنی ۷۰۰ KB، فقط به ۸۰ ms می‌رسد. رشد زمان اجرا در این الگوریتم بسیار آرام است و به ازای هر ۱۰۰ KB تقریباً ۵ ms افزایش پیدا می‌کند. میانگین زمان اجرای آن ۶۵ ms و انحراف معیار آن ۱۰ ms است که نشان‌دهنده پایداری بسیار خوب الگوریتم است.
- **الگوریتم Alg.2:** این الگوریتم در شروع، زمان اجرای بیشتری نسبت به دو الگوریتم دیگر دارد و برای داده ۱۰۰ KB از ۲۰۰ ms شروع می‌شود. با این حال، شیب رشد آن نسبتاً ملایم است و در هر گام فقط ۲۰ ms افزایش دارد. میانگین زمان اجرای این الگوریتم برای کل داده‌ها برابر ۲۶۰ ms است. بنابراین، اگرچه شروع آن کندتر است، اما در داده‌های بزرگ‌تر رفتار قابل قبول‌تری نسبت به الگوریتم سوم دارد.

• **الگوریتم Alg.3:** این الگوریتم برای داده کوچک ۱۰۰ KB با زمان ۱۰۰ ms عملکرد بهتری نسبت به Alg.2 دارد، اما مشکل اصلی آن نرخ رشد زیاد زمان اجراست. زمان اجرای آن در هر گام حدود ۱۰۰ ms افزایش می‌یابد و در داده ۷۰۰ KB به ۷۰۰ ms می‌رسد. میانگین زمان اجرای آن ۴۰۰ ms و انحراف معیار آن ۲۰۰ ms است که نشان می‌دهد این الگوریتم نسبت به افزایش حجم داده حساسیت زیادی دارد.

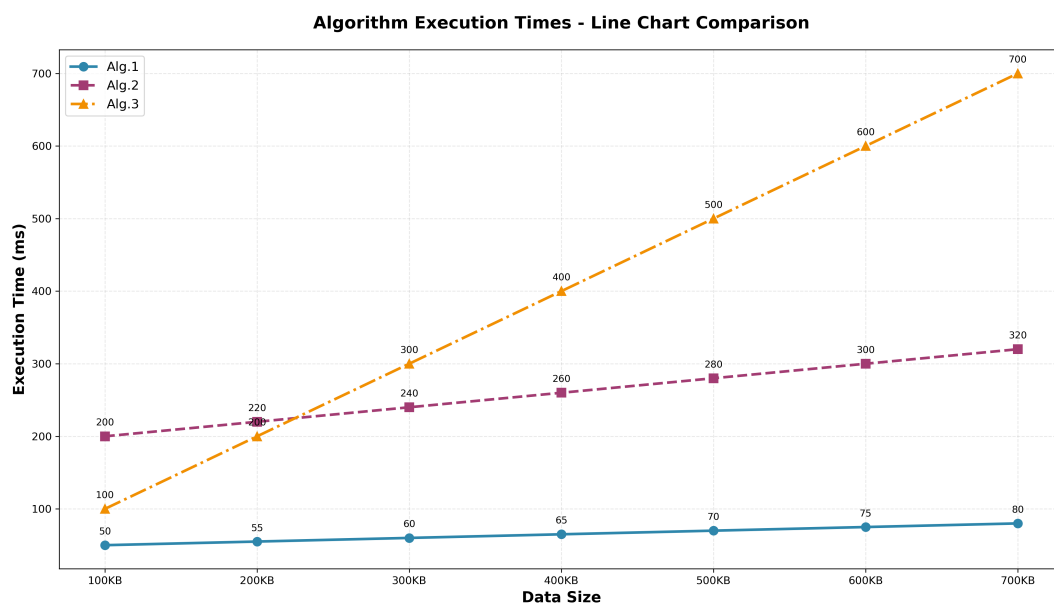
یک نکته آماری جالب این است که چون زمان اجرای هر سه الگوریتم روندی کاملاً خطی و منظم دارد، مقدار میانگین (Mean) و میانه (Median) برای هر الگوریتم با هم برابر شده است.

بررسی نمودارهای ترسیم‌شده

برای تحلیل دقیق‌تر، سه نمودار اصلی رسم شد.

۱. نمودار خطی (Line Chart)

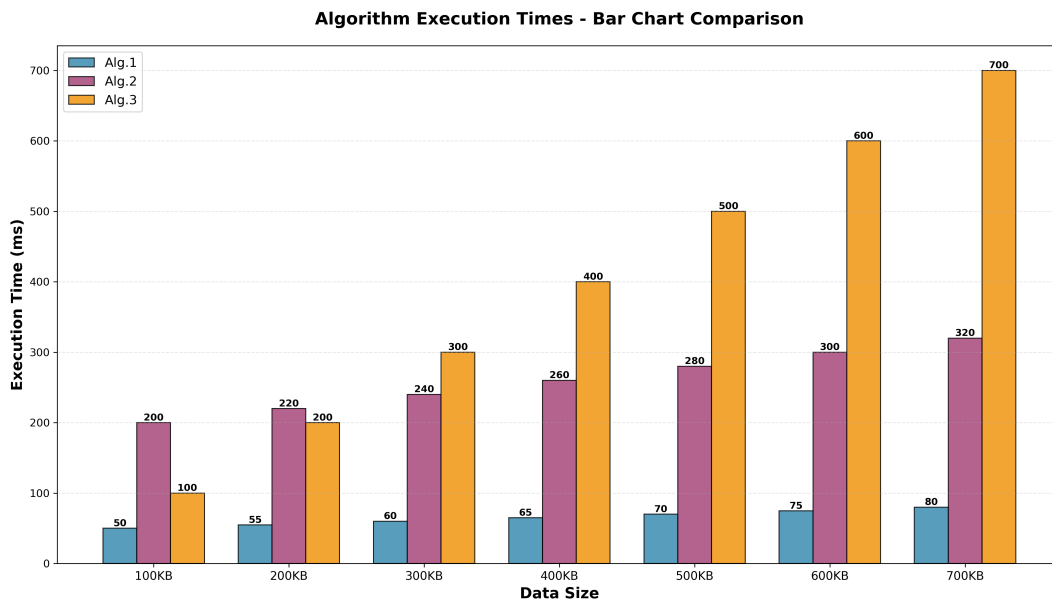
در شکل ۲ روند رشد زمان اجرای هر سه الگوریتم دیده می‌شود. مهم‌ترین نکته در این نمودار، برخورد خطوط Alg.2 و Alg.3 در اندازه ۲۰۰ KB است. این نقطه نشان می‌دهد که برای داده‌های کوچک‌تر از ۲۰۰ کیلوبایت، الگوریتم سوم می‌تواند گزینه بهتری باشد؛ اما از آن نقطه به بعد، الگوریتم دوم به دلیل شیب رشد کمتر، عملکرد مناسب‌تری دارد.



شکل ۲: نمودار خطی مقایسه زمان اجرای الگوریتم‌ها بر حسب اندازه داده

۲. نمودار میله‌ای (Bar Chart)

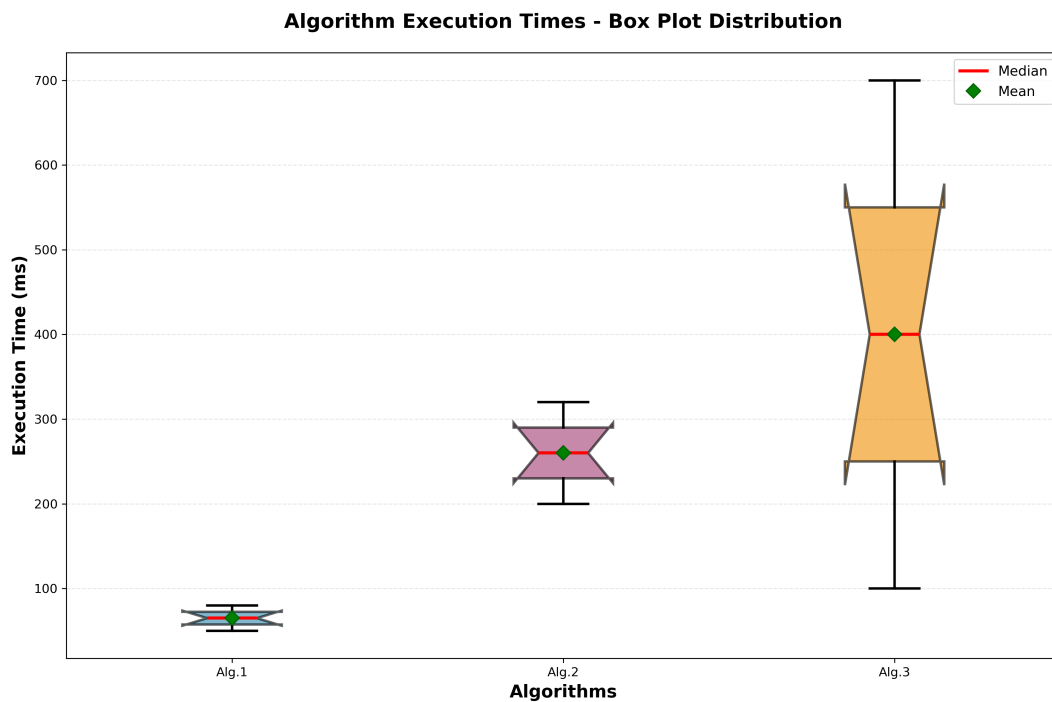
شکل ۳ مقایسه نقطه‌به‌نقطه الگوریتم‌ها را نشان می‌دهد. در این نمودار به‌خوبی مشخص است که ستون‌های مربوط به Alg.3 با افزایش اندازه داده، خیلی سریع‌تر از دو الگوریتم دیگر رشد می‌کنند. در انتهای نمودار، اختلاف این الگوریتم با دو روش دیگر کاملاً محسوس است.



شکل ۳: نمودار میله‌ای زمان اجرای الگوریتم‌ها به تفکیک اندازه داده

۳. نمودار جعبه‌ای (Box Plot)

شکل ۴ برای بررسی پراکندگی زمان اجرا رسم شده است. کوچک بودن جعبه مربوط به Alg.1 نشان می‌دهد که زمان اجرای این الگوریتم در اندازه‌های مختلف داده تغییر زیادی ندارد. در مقابل، کشیدگی زیاد جعبه Alg.3 بیانگر پراکندگی بالا و حساسیت زیاد آن نسبت به حجم داده ورودی است. همچنین قرار گرفتن میانگین روی میانه در هر سه نمودار، خطی و متقارن بودن داده‌ها را تأیید می‌کند.



شکل ۴: نمودار جعبه‌ای پراکندگی زمان اجرای الگوریتم‌ها

ج) محاسبه میانگین زمان اجرای الگوریتم ۲ در بازه ۱۰۰ تا ۶۰۰ کیلوبایت

در این بخش فقط داده‌های مربوط به الگوریتم ۲ برای اندازه‌های ۱۰۰ KB تا ۶۰۰ KB در نظر گرفته شده است؛ بنابراین سطر مربوط به ۷۰۰ KB در این محاسبه وارد نمی‌شود.

مقادیر استخراج شده از فایل برابرند با:

۲۰۰, ۲۲۰, ۲۴۰, ۲۶۰, ۲۸۰, ۳۰۰

میانگین این مقادیر به صورت زیر محاسبه می‌شود:

$$\text{Average} = \frac{۲۰۰ + ۲۲۰ + ۲۴۰ + ۲۶۰ + ۲۸۰ + ۳۰۰}{۶} = \frac{۱۵۰۰}{۶} = ۲۵۰ \text{ ms}$$

این مقدار در خروجی برنامه پایتون نیز دقیقاً برابر ۲۵۰ ms به دست آمده و با محاسبه دستی کاملاً مطابقت دارد.

سوال ۳: محاسبه عددی انتگرال و مقایسه روش‌ها

در این بخش، هدف محاسبه مقدار انتگرال معین تابع $f(x) = e^{x^2}$ در بازه $[0, 1]$ است. این انتگرال جواب ساده‌ای بر حسب توابع مقدماتی ندارد؛ بنابراین باید مقدار آن را با روش‌های عددی تقریب بزنیم. مقدار مرجع انتگرال برابر است با:

$$۱.۴۶۲۶۵۱۷۴۵۹۰۷$$

این مقدار به عنوان مبنای مقایسه در نظر گرفته شده و خطای مطلق هر روش نسبت به آن محاسبه شده است.

نحوه پیاده‌سازی

برای محاسبه عددی انتگرال

$$f(x) = e^{x^2}$$

در بازه

$$[0, 1]$$

از سه روش استفاده شد:

۱. روش ذوزنقه‌ای مرکب: در این روش، بازه انتگرال‌گیری به n زیربازه مساوی تقسیم شد و مقدار انتگرال روی هر زیربازه با تقریب ذوزنقه‌ای به دست آمد. این روش پیاده‌سازی ساده‌ای دارد، اما در مقایسه با روش‌های دقیق‌تر، برای رسیدن به خطای کم به تعداد زیربازه بیشتری نیاز دارد.

۲. روش سیمپسون $3/1$ مرکب: در روش سیمپسون، به جای تقریب خطی، از تقریب سهمی‌وار استفاده می‌شود. این روش فقط زمانی قابل استفاده است که تعداد زیربازه‌ها زوج باشد. دقت سیمپسون معمولاً از روش ذوزنقه‌ای بیشتر است و با تعداد تقسیم‌بندی کمتر می‌تواند جواب دقیق‌تری تولید کند.

۳. روش گاوس-لژاندر: در روش گاوس-لژاندر، نقاط نمونه‌برداری و وزن‌ها به صورت بهینه انتخاب می‌شوند. ابتدا گره‌ها و وزن‌ها روی بازه استاندارد محاسبه شده و سپس به بازه $[0, 1]$ تبدیل می‌شوند. مزیت مهم این روش این است که با تعداد کمی ارزیابی تابع، می‌تواند به دقت بسیار بالایی برسد.

مقدار مرجع و مقایسه خطا: چون این انتگرال پاسخ بسته ساده‌ای ندارد، برای محاسبه از `scipy.integrate.quad` استفاده شد. سپس خروجی هر سه روش با مقدار مرجع مقایسه و خطای مطلق آن‌ها محاسبه شد.

تحلیل همگرایی: نتایج نشان می‌دهد که روش گاوس-لژاندر با تعداد ارزیابی کمتر، دقیق‌ترین نتیجه را تولید می‌کند. همچنین روش سیمپسون نسبت به روش ذوزنقه‌ای سریع‌تر همگرا می‌شود؛ زیرا مرتبه خطای آن

$$O(h^4)$$

است، در حالی که مرتبه خطای روش ذوزنقه‌ای برابر است با:

$$O(h^2)$$

الف) بررسی روش‌های نیوتن-کوتس، دوزنقه‌ای و سیمپسون

روش‌های دوزنقه‌ای (Trapezoidal) و سیمپسون (Simpson) از خانواده فرمول‌های نیوتن-کوتس هستند. در این روش‌ها، بازه انتگرال‌گیری به n زیربازه مساوی با طول گام h تقسیم می‌شود.

- روش دوزنقه‌ای مرکب: در این روش، تابع با خط‌های راست، یعنی چندجمله‌ای‌های درجه ۱، تقریب زده می‌شود. خطای نظری این روش از مرتبه $O(h^2)$ است. نتایج عددی نیز همین موضوع را نشان می‌دهند. برای مثال، وقتی تعداد زیربازه‌ها از $n = 16$ به $n = 32$ می‌رسد، خطای مطلق از 1.77×10^{-3} به 4.42×10^{-4} کاهش پیدا می‌کند. نسبت این دو خطا تقریباً ۴ است و با رابطه $2^2 = 4$ هماهنگی دارد. در $n = 128$ خطا به 2.77×10^{-5} می‌رسد، اما همچنان برای رسیدن به دقت‌های خیلی بالا نیاز به زیربازه‌های بیشتری دارد.

- روش سیمپسون مرکب: در روش سیمپسون، تابع با سهمی‌ها، یعنی چندجمله‌ای‌های درجه ۲، تقریب زده می‌شود. خطای نظری این روش از مرتبه $O(h^4)$ است. طبق نتایج، با دو برابر شدن n از ۱۶ به ۳۲، خطا از 4.58×10^{-6} به 2.88×10^{-7} می‌رسد. نسبت کاهش خطا تقریباً ۱۶ است که با رابطه $2^4 = 16$ مطابقت دارد. این روش با همان $n = 128$ به خطای بسیار کوچک 1.13×10^{-9} رسیده و از روش دوزنقه‌ای به مراتب دقیق‌تر است.

ب) بررسی روش گاوس-لژاندر

در روش‌های نیوتن-کوتس، نقاط نمونه‌برداری فاصله‌های مساوی دارند؛ اما در روش گاوس-لژاندر (Gauss-Legendre) هم محل نقاط و هم وزن‌ها به شکل بهینه انتخاب می‌شوند. یک فرمول گوسی n نقطه‌ای می‌تواند انتگرال چندجمله‌ای‌هایی تا درجه $2n - 1$ را دقیق محاسبه کند.

نتایج این روش بسیار قابل توجه است. تابع $f(x) = e^{x^2}$ تابعی هموار (Smooth) است و به همین دلیل روش گاوس-لژاندر برای آن بسیار سریع همگرا می‌شود. در واقع، این روش فقط با ۸ نقطه ارزیابی به خطای 1.02×10^{-13} رسیده است؛ خطایی که عملاً در محدوده دقت ماشین (Machine Precision) قرار دارد.

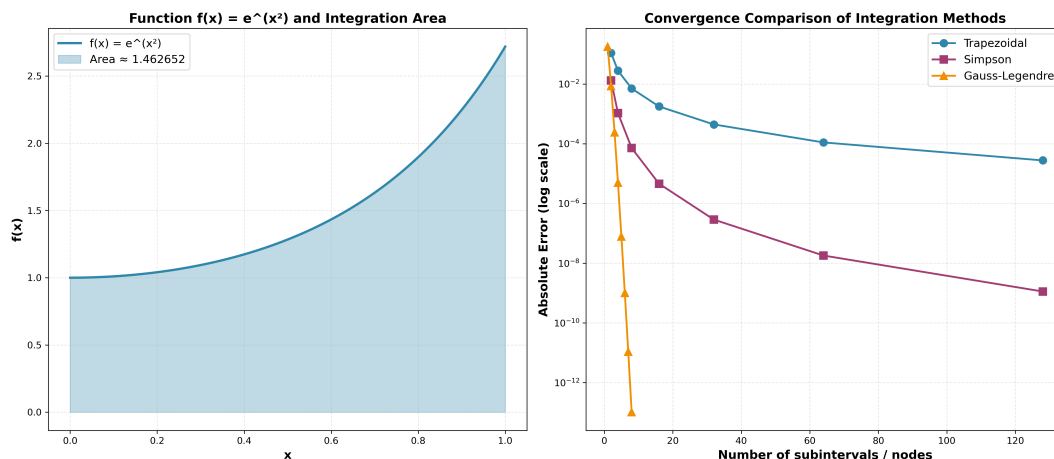
این نتیجه نشان می‌دهد که برای توابع هموار، روش گاوس-لژاندر می‌تواند با هزینه محاسباتی بسیار کمتر، دقتی بسیار بالاتر از روش‌های دوزنقه‌ای و سیمپسون ارائه دهد.

پ) تحلیل نمودارها و نتیجه‌گیری

برای مقایسه بهتر روش‌ها، دو نمودار در شکل ۵ رسم شده است.

- نمودار سمت چپ: این نمودار تابع $f(x) = e^{x^2}$ و مساحت زیر منحنی آن را در بازه $[0, 1]$ نشان می‌دهد. از شکل تابع مشخص است که نمودار در این بازه صعودی و محدب است.
- نمودار سمت راست: این نمودار روند کاهش خطای مطلق روش‌ها را در مقیاس لگاریتمی نمایش می‌دهد. محور افقی تعداد نقاط یا زیربازه‌ها و محور عمودی خطای مطلق است. خط مربوط به روش دوزنقه‌ای کاهش

آرام‌تری دارد، خط سیمپسون با شیب بیشتری پایین می‌آید، اما روش گاوس-لژاندر افت بسیار سریع‌تری را تجربه می‌کند. این رفتار نشان می‌دهد که در این مسئله، با اضافه شدن تعداد کمی نقطه، خطای روش گاوسی به سرعت کاهش می‌یابد.



شکل ۵: چپ: مساحت زیر منحنی تابع. راست: مقایسه لگاریتمی خطای روش‌های انتگرال‌گیری

نتیجه‌گیری: برای تابعی هموار مانند e^{x^2} ، روش گاوس-لژاندر بهترین عملکرد را دارد؛ چون با تعداد کمی ارزیابی تابع، به دقت بسیار بالایی می‌رسد. با این حال، اگر داده‌ها به شکل جدولی و با فاصله‌های مساوی داده شده باشند و تابع تحلیلی در دسترس نباشد، روش سیمپسون گزینه‌ای بسیار مناسب و دقیق خواهد بود. روش دوزنقه‌ای نیز ساده و قابل فهم است، اما برای رسیدن به دقت بالا معمولاً به تعداد تقسیم‌بندی بیشتری نیاز دارد.